

The Question Is a Region: A Neurosymbolic Retrofit for Categorical Question Answering over Rule Engines

Liana Hadarean HADAREAN@AMAZON.COM
Martin Schäf SCHAEF@AMAZON.COM

Tancrède Lepoint TLEPOINT@AMAZON.COM
Willem Visser VISSIE@AMAZON.COM

Amazon Web Services

Editors: Alessandra Mileo, Andrea Passerini and Cogan Shimizu

Abstract

Many consequential decisions, from tax liability and travel reimbursement to benefit eligibility and building-code compliance, are governed by rule engines that map a fully specified set of typed input variables to a finite set of output categories. To retrofit these engines for agentic or chatbot settings, we must relax the requirement of a complete input: users rarely ask about one fully specified case. They ask about regions of the input space, leaving variables under-specified that they do not yet know or do not consider relevant.

Neither a language model nor a symbolic enumeration answers regional questions completely on its own. A language model gives no completeness guarantee, and the gap widens as the decision space grows. A symbolic enumeration finds every path, but a large partition cannot be summarized faithfully. Our method resolves both: we enumerate the paths symbolically, and when the partition is too large to state faithfully, we ask the one follow-up question that removes the most cases. We prove the partition complete by reconstructing the engine’s own path constraints in an SMT solver, so completeness is a checked theorem rather than a claim about the path enumerator, and the budget keeps the presented answer small and its verification cheap. The result is a natural conversational interface: instead of an overwhelming partition, the user gets a meaningful follow-up that drives the dialog to a complete answer.

We demonstrate the approach by retrofitting CATALA, a domain-specific language for computational law. CATALA provides a concolic execution engine, CUTEcat, which we drive to enumerate the paths, wrapped in an LLM agent that lifts the question to a region and verbalizes the cited answer. We evaluate on open-source encodings of French and United States legal texts. The system returns complete, cited partitions where a one-shot language model given the same rules does not; every verdict comes from the unmodified engine. Completeness is a checked theorem for the region the lift produces, and only the lift from question to region is unverified.

1. Introduction

Many consequential decisions are governed by rule engines hidden behind complex web forms: tax filing, mortgage calculators, insurance eligibility, and e-government benefit applications. These forms require the user to fill in every field before the engine returns an answer. In the form setting that works, because the interface guides the user through each field in sequence.

Retrofitting these engines for chat settings breaks this model. A chat interface cannot present thirty required fields as a form, and asking follow-ups until every field is specified

is a poor experience: the user rarely knows every value, the interrogation is long, and the final single-point answer may miss what the user wanted. Many questions are not about one fully specified case at all. They ask about *regions* of the input space: “*can I deduct home-office expenses if I work remotely part of the week?*”, “*am I eligible for parental leave if my contract is fixed-term?*”, “*when does my child qualify for family benefits?*” The user pins a few facts and leaves the rest open. The right answer is not one verdict but a set of conditions over the remaining variables: yes under these conditions, no under those, citing the rules that decide each case.

Our approach. We propose a novel method for retrofitting existing rule engines into conversational settings that requires no changes to the engine itself. We read the engine’s scopes, input variables, and type definitions to route and lift questions, and we drive a symbolic execution of the rules without modifying them. Our wrapper accepts *symbolic inputs*, where mentioned variables are pinned and the rest stay free or loosely constrained, and resolves the symbolic state into a small set of concrete inputs that together cover every reachable output category. Each concrete input is passed to the original engine, so every verdict is the engine’s own output.

The loop has three stages, and none is tied to a particular engine. First, *lift*: a language model reads the question and produces a symbolic input, constraining the variables the user stated and leaving the rest free. Second, *cover*: a symbolic analysis explores every path through the rules consistent with that input. Each path is a distinct combination of rule firings, so the analysis enumerates the output categories reachable along these paths and produces one concrete witness per category, which the unmodified engine adjudicates. It runs to the symbolic engine’s path exhaustion rather than sampling, and its size is bounded by the rules that read the free variables, not the cross-product of their domains, so it stays tractable even when the input space is large.

Third, *clarify*. A large partition is costly on three counts. A long list of conditions is hard for a person to read. The model that verbalizes it is more likely to drop a branch as the list grows. And the symbolic engine does more work, since it enumerates and adjudicates more paths. So when the partition exceeds a budget K , we neither dump every case nor guess. For each free variable we measure how much constraining it would collapse the partition, pick the one with the largest reduction, and verbalize it as a single follow-up question. The user’s answer pins that variable, and the loop runs again over the smaller region. Each round constrains the input a little more, until the categories fall within K , which we can present and verify. The symbolic structure chooses which variable to ask about, and the model phrases the question. So the loop steers an under-specified question toward one the engine can answer in full. At $K = 1$ the loop reduces to the naive system that asks the user to fill in every field; for any larger K it stops as soon as the answer is small enough to state in full, usually after far fewer questions.

We demonstrate the approach on CATALA (Merigoux et al., 2022), a domain-specific language for computational law. CATALA ships open-source encodings of real French and United States statutes as *literate* programs that pair each legal text with its executable rules. This pairing gives a clean baseline: we put the same source in a language model’s context and compare its answers to ours. For the symbolic analysis we use CUTEcAT (Goutagny et al., 2025), a concolic execution engine CATALA already provides: it returns each path’s condition

as an SMT formula and evaluates a symbolic input down to a concrete witness. We reuse those SMT path conditions to prove the partition complete in z3, inheriting the engine’s own encoding instead of re-deriving the rules. The same loop fits other policy formats, such as the Cedar authorization language (appendix A) and Horn-clause engines, but we lack shareable open-source policies for them, so our public evaluation uses CATALA. Section 2 describes the per-turn loop, and section 3 evaluates the system on 13 under-specified questions across the four CATALA statutes of CUTECat’s benchmark suite, comparing the z3-verified cited partitions it returns against a language model given the same rules and against one that can call the engine itself. Section 4 positions the work against symbolic execution, rule-base analysis, and neurosymbolic QA, and section 5 concludes with limitations and scope for future work.

2. Approach

Figure 1 outlines the approach. A one-time, deterministic step (top) reads the engine’s scopes, input variables, and types and builds a catalog of askable outputs. Each user turn then runs a loop (bottom). The LLM *lifts* the question to a region of pinned and free variables. We *harness* that region as a small program that pins the stated variables and exposes the rest as free inputs. A symbolic engine *covers* the region, enumerating one concrete input per reachable execution path and running each on the unmodified engine. We *cluster* the resulting witnesses by output into a cited conditional answer, and either *render* it or, when too many categories remain, *clarify* by asking the one follow-up that collapses the most cases, then repeat. The cover engine is CUTECat (Goutagny et al., 2025), used unmodified; the engine is the oracle for every verdict; the LLM only lifts and verbalizes. Our contribution is the wrapper around the engine: lifting an under-specified question to a region, harnessing it so the symbolic engine enumerates exactly that region, clustering the witnesses into a cited partition, and choosing the follow-up that most reduces an over-large one.

Rules and output categories. A CATALA *scope* computes one or more outputs from a fixed set of typed *input variables* $V = \{v_1, \dots, v_n\}$, each with a domain D_i (Boolean, enumeration, integer, money, or date). CATALA resolves defaults and exceptions internally (Lawsky, 2018; Merigoux et al., 2022); we treat the engine as a black box and read its result. For a Boolean or enumeration output the categories are its values, a finite set C . A numeric output ranges over an infinite domain; there a *category* is a *functional piece*: a maximal region of the input space on which a single execution path determines the output formula. Either way, what is finite, and what the cover enumerates, is the set of reachable execution paths, not the output values. A free integer variable does not produce infinitely many paths, because each branch condition partitions its axis into finitely many intervals.

The artifact we return is a *conditional answer*: a set of branches, each pairing an output category y_i with a condition c_i over the unspecified variables and a citation ρ_i to the statute article that decides it, read as “ y_i if c_i , per ρ_i .” When every branch carries the same verdict the answer collapses to that single category; otherwise the overall verdict is **It depends**.

Our running example is the French minimum wage (fig. 2). A CATALA rule set sets the hourly rate from the date and the territory of residence: a national base rate, revised over time, with a lower rate in Mayotte. Asking for the wage without fixing the date or the

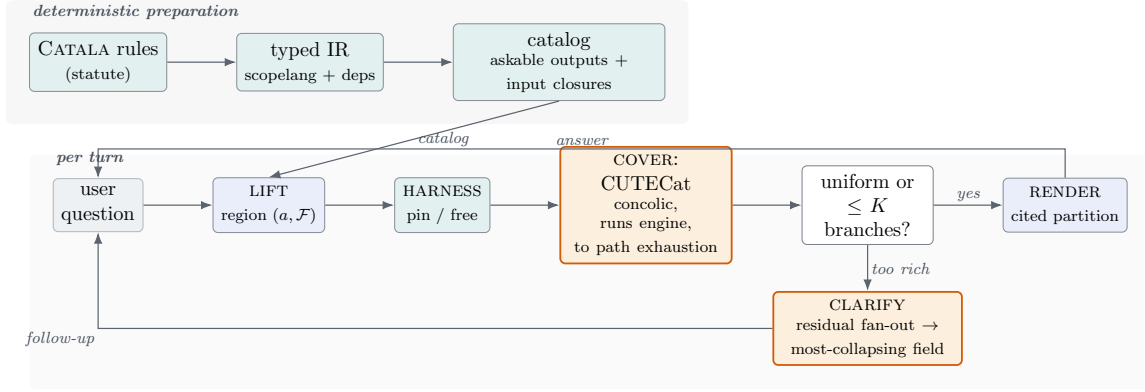


Figure 1: The retrofit end to end. A one-time deterministic step (top) reads the engine’s scopes, variables, and types and builds a catalog of askable outputs. Each turn (bottom) lifts the question to a region, harnesses it, covers the reachable categories by running CUTEcCat (Goutagny et al., 2025) to path exhaustion, and clusters the witnesses into a cited partition; when too many categories remain, it asks the one follow-up with the largest fan-out reduction and repeats. The cover engine is CUTEcCat, used unmodified. The engine is the oracle for every verdict; the LLM only lifts and verbalizes.

territory leaves both free, so the question covers a region with many categories, one per distinct rate the rules produce.

Lift. A corpus defines many outputs, so the lift first *routes*. A one-time step reads the engine’s structure (for CATALA, the `scopelang` listing and the dependency graph) and builds a catalog of askable outputs, each with its statute article and the typed closure of inputs that feed it. The model selects one output by its description and lifts onto that closure alone. The lift is a single constrained tool call: the schema pins each enumeration field to its exact domain, so the model cannot emit a value the type lacks, and it records only fields the question states, never inferring one from context. The result is a *symbolic state* $(a, \mathcal{F}, \varphi)$: a pins the stated variables to values, $\mathcal{F}$ is the set of input variables left free, and φ collects loose bounds the user gave (for example, “earnings above €1000”). Every variable the lift does not record stays free. Routing ranges over the catalog and the lift over one closure, so the prompt stays small however many rules the corpus holds. Because the lift maps surface phrases onto fixed identifiers, it handles questions posed in different natural languages.

Cover. Given the symbolic state we want every output category the engine can still reach. We build a *harness*: a small CATALA scope that calls the rule set with each pinned variable set to its value and each free variable exposed as an input, and we run CUTEcCat (Goutagny et al., 2025) on it. CUTEcCat is a concolic (concrete plus symbolic) execution engine: it runs the program on concrete inputs while collecting symbolic path conditions, then negates branches to cover new paths until exhaustion. CUTEcCat explores the harness to path

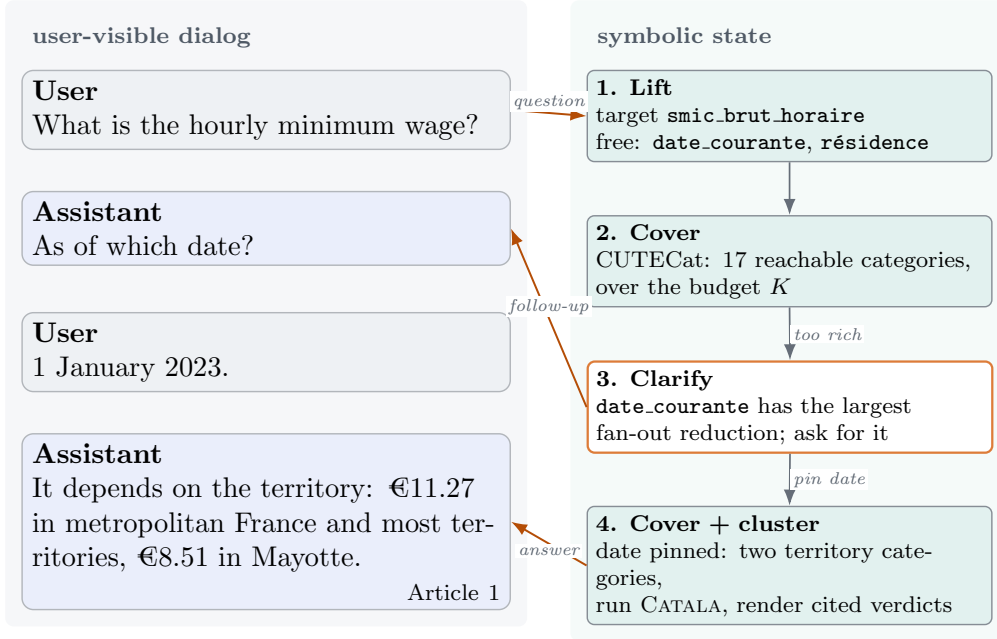


Figure 2: An example interaction with the agent of fig. 1, on the French minimum wage. The date and territory are left open, so the cover exceeds the budget; the agent asks for the date, then answers by territory, each branch cited.

exhaustion, returning one concrete input per reachable path; we run each on the unmodified interpreter, so the output attached to it is the engine’s own. The SMT solver that finds these inputs is internal to CUTECat; we call no solver to build the cover, and invoke z3 ourselves only for the completeness check below. Path exhaustion is what makes the cover complete: CUTECat reports every reachable path under the region rather than a sample, and signals when it has done so. The number of paths is bounded by the rules whose inputs meet a free variable, not by the product of the variable domains, which keeps the cover small even when the raw input space is astronomically large. A pinned variable holds a fixed value and does not branch.

Cluster into a cited answer. CUTECat returns a set of witnesses, each a concrete input, the engine’s output on it, and the execution path it took. We group the witnesses by their *symbolic output*, the formula CUTECat computes on the path, rather than by the concrete witness value. For an enumeration this formula is a constant, the selected value; for a numeric output it is the functional piece the path computes, for instance the discount `customer_price` – `aggregate_cost` on the gross-profit-cap path of Section 132. Grouping by the piece partitions a continuous output by its pieces, instead of collapsing it onto whichever value a witness happened to hit. For each branch we render its condition from the path’s SMT constraint, simplified and projected onto the still-free variables (for instance `discount_type` property and `customer_price` $\geq$ `aggregate_cost`), and we cite the article of the rule on that path. An error outcome, such as a division by zero or an

unresolved conflict, forms its own branch and is surfaced rather than hidden. Because each piece is computed by a distinct combination of rules, grouping by symbolic output also separates branches by the rule that decides them, with the citation naming it. The piece and the condition both come from the engine’s own execution, so the explanation cannot name a test the engine did not perform.

Faithfulness and completeness. Two things must hold for the answer to be trustworthy, and we check both against the engine. First, every verdict is the unmodified engine’s own: CUTEcat runs each witness on the interpreter, and citations come from re-running it under `catala interpret --trace`, which logs the article of each rule the engine applied. We build no SMT model of the statute. We read CUTEcat’s own path constraints and output formulas, so our coverage inherits the engine’s encoding and there is no second formalization to keep faithful. Second, completeness is not taken on trust. Each path carries its constraint as an SMT formula over the inputs; we reconstruct these in z3 (de Moura and Bjørner, 2008), declaring each free variable by its type (money and dates as integers at cent and day precision, an enumeration as a finite datatype), and prove the path regions are pairwise disjoint and that their disjunction is valid ($\neg \bigvee_i R_i$ unsatisfiable), so they tile the whole input space. The engine’s own paths, re-expressed in z3, are proved to leave no input uncovered, so completeness is a theorem about the partition rather than a property assumed of the path enumerator. When a rule set uses a construct CUTEcat evaluates concretely rather than symbolically, notably unbounded lists (Goutagny et al., 2025), we mark the affected branch reduced-confidence.

Render, clarify, and iterate. If all paths map to one category the answer is uniform and we respond at once. If the categories number at most a presentation budget K , we render the whole partition: each branch as a condition over the still-free variables with the engine’s verdict and citation, read as “€11.27 in metropolitan France, per Article 1,” and we drop any clause that restates a fact the user gave. Otherwise we ask one follow-up. To choose it, picture the reachable categories as the leaves of a decision tree whose nodes test the free variables. Each free variable splits the leaves into groups, one group per value it can take, and the user’s answer keeps one group and drops the rest. We ask about the variable whose largest group is the smallest. Then whichever value the user gives, the categories left are as few as possible, and the dialog converges in a couple of turns. This ranking is read off the single cover we already ran, with no re-enumeration and no further engine calls, so choosing the question is free. The reply becomes a new pin, the free set shrinks, and the turn repeats until the categories fall within K . The decision to present or to ask is the wrapper’s, taken from the partition size, not the model’s; the model only relays the branches. Figure 2 traces this: the wage question leaves the date and territory free, so the cover reaches seventeen categories; the date is the variable whose largest group is smallest, so the system asks for it, and the second turn renders the two remaining categories by territory, each cited.

3. Evaluation

Our evaluation addresses three questions:

- **RQ1 (Lift fidelity).** Does the LLM lift recover the intended region, that is, the correct pinned and free variables, from a natural-language question?

- **RQ2 (Validity and completeness).** Is every branch the system reports correct and sufficient, verified by executing the engine, and is the partition complete, proved in z3? How do an LLM given the same rules, and an LLM that can call the engine itself, compare?
- **RQ3 (Clarification).** When the partition exceeds the presentation budget, does the residual-fan-out follow-up reduce it to a complete answer, and in how many turns?

3.1. Setup

Rule sets. We evaluate on the four CATALA rule sets that make up CUTEcat’s own benchmark suite (Goutagny et al., 2025; CatalaLang, 2024), so the rules and their reachable partitions are studied by others and not authored by us. They span jurisdiction, language, and output shape: Section 132 of the United States tax code (`QualifiedEmployeeDiscount`, a discount capped differently for property and services); the French minimum wage (`Smic`, indexed by date and territory); the French family quotient (`nombre_de_parts`, the number of tax-household parts implied by the household’s structure); and the French housing benefits (`aides_logement`, the largest CATALA codebase at roughly 19,000 lines). We use the input variables from CUTEcat’s benchmark suite for each rule set. Freeing all of them, CUTEcat enumerates 7, 17, 18, and 75 reachable paths on the four rule sets, complete in each case. The benchmarks below free subsets of these inputs. We use this benchmark deliberately. Its encodings are validated against the law by the Catala project, so the partitions are trustworthy ground truth, not rules chosen to flatter the method. The set is small on purpose: the lift and baseline are the only LLM-dependent steps, so we read every question and answer by hand, checking that no question names a free variable or encodes the partition and that answers are scored only by re-running on the engine. For scale we turn to the Cedar instantiation (appendix A), which tests arbitrary policies against a large cell-level gold: Catala supplies qualitative depth, Cedar supplies scale.

Benchmarks and gold. From each rule set we fix one base scenario and free a subset of its variables, giving 13 benchmarks (four for Section 132, three each for the others) with one to three free variables; the freed subset is the *region*. An LLM turns each benchmark into one natural-language question from the output description and the pinned facts, never naming the freed variables, so the region stays implicit; the 13 questions are fixed once, so the freed subset is ground truth for the lift. The gold partition is CUTEcat’s path-exhaustive cover, with its per-path symbolic outputs grouped into functional pieces (2 to 17 per benchmark) and proved disjoint and complete in z3 (section 2), on all 13 with no atom unmodeled. The baselines are scored by recall against this partition.

Scoring and baselines. We score by re-running on the engine. *Lift fidelity*: did the agent free the intended variables? *Validity*: does the engine confirm each reported branch? *Completeness*: fraction of gold categories covered, judged post-follow-up. We compare two baselines, both Claude Sonnet 4.5 at temperature 0.¹ *B1 (pure-LLM)* gets the rules in context and no engine. *B2 (engine-using)* also gets a tool that runs `catala interpret` on one concrete input, so it can probe the engine but has no symbolic coverage. At temperature 0,

1. Anthropic API, `claude-sonnet-4-20250514`. Prompts for the lift, verbalization, and baselines are available in the supplementary material.

Table 1: Per-rule-set results, budget $K = 4$. $|P|$: reachable paths with all exposed inputs free (the benchmarks free subsets). “Valid”: fraction of reported witnesses that reproduce on the engine. “Complete (z3)”: benchmarks with a proven-complete partition. “FU”: clarifying follow-ups asked. Recall B1/B2: fraction of the proven partition recovered by B1 (pure-LLM) and B2 (engine-using), median of three trials per question.

Rule set	Shape	#Q	$ P $	Valid	Complete (z3)	Recall B1	Recall B2	FU
US tax §132	categ./thresh.	4	7	1.00	1.00	0.25	0.25	0
SMIC	date/enum	3	17	1.00	1.00	0.94	0.63	2
Family quotient	counts	3	18	1.00	1.00	0.28	0.56	3
Housing benefits	money/enum	3	75	1.00	1.00	0.00	0.78	1
Total		13		1.00	13/13	0.36	0.53	6

B1 produced identical outputs across all three trials; B2 varied on one question (SMIC with all inputs free, where two trials returned no answer).

3.2. Results and discussion

Table 1 reports the results.

RQ1 (lift). The lift recovered the intended region on 12 of the 13 questions, the same result in every run at temperature 0, across the English rule set and the three French ones and across enumeration, money, integer, and date axes. Each question targets a known scope, so this isolates variable selection from routing, which we do not separately evaluate (an intentional scope choice: routing selects among catalog entries, and our questions target only one output each, so the evaluation measures lift fidelity in isolation; evaluating routing over corpora with multiple plausible targets is left to future work). The one miss frees both the discount type and the customer price while stating the aggregate cost. That cost applies only to property, so the lift infers `discount_type = property` and fixes the otherwise-free enum. This is the failure mode the trust model is designed around: the symbolic guarantees hold for the region the lift produces, but the lift itself is outside the verified core. Detecting whether a region matches user intent is a separate problem. The lift held up here, so we did not stress it further; a dedicated lift study with adversarial paraphrases is out of scope and left to future work.

RQ2 (validity and completeness). Every branch our system reports is valid: it is a CUTEcat execution path, and all 76 of 76 presented witnesses reproduce on `catala interpret`, so validity is established on the engine, not assumed. Completeness is an SMT theorem. Reconstructing the path constraints in z3, the partition is proved disjoint and to tile the input space on all 13 benchmarks, with no atom left unmodeled. This is an independent check, not the earlier by-construction argument.

Neither baseline matches the cover (table 1). Reading the two output regimes explains why. On the *categorical* outputs (7 questions: the territory wage, the housing zone and family situation, the §132 property-or-service case), B2 recovers more of the partition than

B1, recall 0.75 versus 0.48: a tool that checks concrete inputs helps. B2 still misses cases, because it probes guessed inputs rather than covering paths, and on the widest question (seventeen SMIC categories) two of its three trials returned no answer at all, which explains why B2 trails B1 on SMIC in table 1. On the *numeric* outputs (6 questions: the §132 discount and the family-quotient parts, each a function of a free variable), both baselines fall short, B1 0.23 and B2 0.28: they emit sample values, or at best note the output is “a range,” while only the symbolic cover yields the functional pieces. The engine in the loop helps the model enumerate discrete outcomes, but only the symbolic cover gives the complete, proven partition, and (section 3.2) it does so more cheaply than letting the model drive the engine.

RQ3 (clarification). At budget $K = 4$, clarification fired on 6 of the 13 questions, the regions with more than four categories: housing benefits over situation and zone (six cases), the minimum wage over date alone (nine) and with territory (seventeen), and the family quotient over the children count (five to seven). In each, the system pinned the free variable of smallest residual fan-out and re-solved, and every dialogue converged after a single follow-up. The baseline, holding the full rule text, issues no clarification. The budget $K = 4$ is just the number of branches we judged readable in one turn; a different K changes only when the system asks, not the guarantees it gives.

Limitations and guardrails. We assume a cooperative user. The questions paraphrase the engine’s own scenarios, so the lift, the one step the engine does not verify, is exercised in-distribution; we did not study adversarial paraphrases. The agent does carry the formal tools to add guardrails as the setting demands. The same SMT encoding that proves completeness can flag *over-* and *under-pinning*, by testing whether freeing a pinned variable splits the region or whether a variable the user stated still leaves the output varying, and it can trigger a clarification when the question omits a fact the rules need. The same lift-and-interrogate step scales to a large budget: we lift the rendered summary back into a region and re-run it through the engine to check the answer we present. We leave these checks to future work.

Cost. The agent makes a few model calls and a CUTEcat-plus-z3 pass per question, a second pass when it asks a follow-up, averaging 37 seconds on rule sets with 7 to 75 reachable paths. B1 is a single large-context call. B2 is the most expensive, averaging 16.5 engine calls and 73 seconds, twice the agent, and still incomplete. On these benchmarks the symbolic cover is both more complete and cheaper than letting the model drive the engine.

4. Related work

Symbolic execution and concolic testing. Symbolic execution covers program paths by collecting path conditions and generating concrete inputs (King, 1976; Godefroid et al., 2005; Sen et al., 2005; Cadar et al., 2008; Cadar and Sen, 2013). CUTEcat (Goutagny et al., 2025) brings concolic testing to CATALA, which we drive unmodified (section 2). The difference is purpose: concolic testing enumerates a whole program’s paths to find bugs and emit a test suite for the implementer, while we constrain the engine to one user’s region and keep its paths as a cited answer.

Rule-base and policy analysis. Expert-system verification, BDD-based rule analysis, and DMN decision-table analysis check global properties (inconsistency, overlap, missing cases) for the rule author (O’Keefe and O’Leary, 1993; Mues and Vanthienen, 2004; Calvanese et al., 2018; Cutler et al., 2024). Law-as-code systems make statutes executable (Sergot et al., 1986; M  rigoux et al., 2021; Merigoux et al., 2022). We assume the executable engine already exists and ask a different question: how should an assistant answer a user who specified only part of the input? Unlike counterfactual explanations (Wachter et al., 2018; Mothilal et al., 2020), which propose nearby changes that flip a single outcome, we return a complete partition over the queried region.

LLMs with solvers. SatLM, LINC, Logic-LM, and ProSLM connect LLMs to SAT/SMT or logic-programming solvers by having the LLM emit a specification that the solver decides (Ye et al., 2023; Olausson et al., 2023; Pan et al., 2023; Vakharia et al., 2024). Auto-formalization translates natural-language policies into logic for verification (Bayless et al., 2026). In these systems the formalization is the critical artifact. We invert the trust boundary: the LLM only lifts the question into an existing schema and verbalizes the result; the engine decides every verdict.

5. Conclusion

A user’s natural-language question to a rule engine is a question about a region of the engine’s input space. The retrofit makes that region explicit: the LLM lifts the utterance and verbalizes the result, a concolic engine enumerates the reachable paths, and the unmodified engine adjudicates each witness. When the partition is too large, the same structure asks the follow-up that shrinks it most. The wrapper groups by symbolic output and proves the partition complete in z3 by reconstructing the engine’s path constraints. Completeness is a checked theorem for the region the lift produces. The lift remains outside the verified core, and the rule encoding is inherited from the engine rather than re-encoded. The activation-literal encoding could also refine per-rule coverage and citations. Against a language model given the same rules, with or without an engine tool, the wrapper is *complete* where the baselines are *partial* (table 1). The contribution is the guarantee, not a higher recall number. The baselines must reason about the rules and enumerate outcomes, while the wrapper only lifts the question and lets the engine do the rest.

Nothing in the loop is specific to CATALA: any engine that exposes its decision logic can be driven the same way, including Cedar through its own symbolic compiler (appendix A) and engines built on Datalog (Ivliev et al., 2024) or Horn clauses (Horrocks and Patel-Schneider, 2004). The retrofit is straightforward because the engine fixes the domain model, so the lift maps a question onto identifiers it already declares rather than inventing a formalization.

We inspected 13 Catala questions and 12 Cedar families. Since they paraphrase engine scenarios, future work should test adversarial phrasing, follow-up clarity, and larger rule sets.

References

- Amazon Web Services. Introducing Cedar, an open-source language for access control. <https://aws.amazon.com/about-aws/whats-new/2023/05/cedar-open-source-language-access-control/>, 2023. AWS What’s New, 10 May 2023.
- AWS Samples. Sample Cedar agentic AI authorization. <https://github.com/aws-samples/sample-cedar-agentic-ai-authorization>, 2026. GitHub repository, accessed 2026-06-03.
- Sam Bayless, Stefano Buliani, Darion Cassel, Byron Cook, et al. A neurosymbolic approach to natural language formalization and verification. In *International Conference on Computer Aided Verification (CAV)*, 2026. arXiv:2511.09008.
- Cristian Cadar and Koushik Sen. Symbolic execution for software testing: three decades later. *Communications of the ACM*, 56(2):82–90, 2013. doi: 10.1145/2408776.2408795.
- Cristian Cadar, Daniel Dunbar, and Dawson R. Engler. KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 209–224, 2008.
- Diego Calvanese, Marlon Dumas, Ülari Laurson, Fabrizio M. Maggi, Marco Montali, and Irene Teinemaa. Semantics, analysis and simplification of DMN decision tables. *Information Systems*, 78:112–125, 2018. doi: 10.1016/j.is.2018.01.010.
- CatalaLang. Catala examples, 2024. URL <https://github.com/CatalaLang/catala-examples>. Reference test suite for the Catala language.
- Cedar Policy Language. Cedar policy language reference guide: What is Cedar? <https://docs.cedarpolicy.com/>, 2026. Accessed 2026-06-03.
- Joseph W. Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Jacobs, Panagiotis Manolios, Andrew McLaughlin, Anwar Nielson, Andrew Schaefer, Max Stuart, Sergey Teleshov, Edwin Westbrook, and Dominik Wojtczak. Cedar: A new language for expressive, fast, safe, and analyzable authorization. *Proc. ACM Program. Lang. (OOPSLA)*, 8, 2024. doi: 10.1145/3649835.
- Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, pages 337–340, 2008. doi: 10.1007/978-3-540-78800-3_24.
- Patrice Godefroid, Nils Klarlund, and Koushik Sen. DART: directed automated random testing. In *PLDI*, pages 213–223, 2005. doi: 10.1145/1065010.1065036.
- Pierre Goutagny, Aymeric Fromherz, and Raphaël Monat. CUTECat: Concolic execution for computational law. In *European Symposium on Programming (ESOP)*, 2025. arXiv:2410.18212. Distinguished Artifact Award.

- Liana Hadarean and Jean-Baptiste Tristan. Why policy in Amazon Bedrock AgentCore chose Cedar for securing agentic workflows. <https://aws.amazon.com/blogs/security/why-policy-in-amazon-bedrock-agentcore-chose-cedar-for-securing-agentic-workflows/>, 2026. AWS Security Blog, 20 May 2026.
- Ian Horrocks and Peter F Patel-Schneider. A proposal for an owl rules language. In *Proceedings of the 13th international conference on World Wide Web*, pages 723–731, 2004.
- Alex Ivliev, Lukas Gerlach, Simon Meusel, Jakob Steinberg, and Markus Krötzsch. Nemo: Your friendly and versatile rule reasoning toolkit. In Pierre Marquis, Magdalena Ortiz, and Maurice Pagnucco, editors, *Proceedings of the 21st International Conference on Principles of Knowledge Representation and Reasoning (KR 2024)*, pages 743–754. IJCAI Organization, 2024. doi: <https://doi.org/10.24963/kr.2024/70>.
- James C. King. Symbolic execution and program testing. *Communications of the ACM*, 19(7):385–394, 1976. doi: 10.1145/360248.360252.
- Sarah B. Lawsky. A logic for statutes. *Florida Tax Review*, 21(1):60–128, 2018.
- Denis Mérigoux, Nicolas Chataing, and Jonathan Protzenko. Catala: A programming language for the law. *Proceedings of the ACM on Programming Languages*, 5(ICFP), 2021. doi: 10.1145/3473582.
- Denis Merigoux, Liane Huttner, and Aymeric Lazrak. Catala: Moving towards the future of legal expert systems. *Artificial Intelligence and Law*, 2022. doi: 10.1007/s10506-024-09418-6.
- Ramaravind K. Mothilal, Amit Sharma, and Chenhao Tan. Explaining machine learning classifiers through diverse counterfactual explanations. In *ACM Conference on Fairness, Accountability, and Transparency (FAT*)*, pages 607–617, 2020. doi: 10.1145/3351095.3372850.
- Christophe Mues and Jan Vanthienen. Efficient rule base verification using binary decision diagrams. In *Database and Expert Systems Applications (DEXA)*, volume 3180 of *LNCS*, pages 445–454, 2004. doi: 10.1007/978-3-540-30075-5_43.
- Robert M. O’Keefe and Daniel E. O’Leary. Expert system verification and validation: a survey and tutorial. *Artificial Intelligence Review*, 7(1):3–42, 1993. doi: 10.1007/BF00849196.
- Theo X. Olausson, Alex Gu, Benjamin Lipkin, Cedegao E. Zhang, Armando Solar-Lezama, Joshua B. Tenenbaum, and Roger Levy. LINC: A neurosymbolic approach for logical reasoning by combining language models with first-order logic provers. In *Empirical Methods in Natural Language Processing (EMNLP)*, 2023. doi: 10.18653/v1/2023.emnlp-main.313.
- Liangming Pan, Alon Albalak, Xinyi Wang, and William Yang Wang. Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. In *Findings of EMNLP*, 2023. doi: 10.18653/v1/2023.findings-emnlp.248.

- Koushik Sen, Darko Marinov, and Gul Agha. CUTE: a concolic unit testing engine for C. In *ESEC/FSE*, pages 263–272, 2005. doi: 10.1145/1081706.1081750.
- M.Ĵ. Sergot, F. Sadri, R.Ā. Kowalski, F. Kriwaczek, P. Hammond, and H.Ĥ. Cory. The British Nationality Act as a logic program. *Communications of the ACM*, 29(5):370–386, 1986. doi: 10.1145/5689.5920.
- Priyesh Vakharia, Olivia Kufeldt, Adam Meyers, Ian Lane, and Leilani Gilpin. ProSLM: A Prolog synergized language model for explainable domain-specific knowledge-based question answering. In *International Conference on Neural-Symbolic Learning and Reasoning (NeSy)*, volume 14980 of *LNCS*, pages 291–304, 2024. doi: 10.1007/978-3-031-71170-1_25.
- Sandra Wachter, Brent Mittelstadt, and Chris Russell. Counterfactual explanations without opening the black box: Automated decisions and the GDPR. *Harvard Journal of Law & Technology*, 31(2):841–887, 2018.
- Andrew M. Wells. What’s analyzable? <https://cedarpolicy.com/blog/whats-analyzable>, 2024. Cedar Policy blog, 13 November 2024.
- Xi Ye, Qiaochu Chen, İřil Dillig, and Greg Durrett. SatLM: Satisfiability-aided language models using declarative prompting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Appendix A. A Cedar/SymCC Instantiation for Authorization Policies

CEDAR is an open-source authorization language for expressing access-control rules outside application code (Cutler et al., 2024; Amazon Web Services, 2023; Cedar Policy Language, 2026). A normal CEDAR call is a point query: the caller supplies a fully specified request—principal, action, resource, context, and entities—and the authorizer returns `Allow` or `Deny`. An operator’s question is often broader: *when may this agent call this tool?* or *which user facts make this request allowed?* Such a question fixes some fields and leaves others open. The region is the set of all requests obtained by filling in those open fields. For example, if principal, action, and resource are fixed but `delegation_depth` is not, the region contains every request with those fixed fields and every possible delegation depth.

CEDAR is a useful second domain because it was designed to be analyzable. Its policies can be reduced to SMT so tools can answer questions such as whether a policy change adds permissions, whether two policy sets are equivalent, or whether some request can ever be allowed (Cutler et al., 2024; Wells, 2024). Thus the CEDAR instantiation does not need a new hand-written policy semantics. It does call for a slightly different approach than the main system. The main system drives CUTEcat, which enumerates a CATALA program’s execution paths over its default logic. CEDAR is not default logic, and its SMT translation through the symbolic compiler works differently from CUTEcat’s path conditions, so instead of enumerating paths we reuse CEDAR’s parser, type checker, authorizer, and symbolic compiler to prove uniform allow/deny branches over regions of authorization requests.

A.1. AgentAuthz running example

The concrete example uses **AgentAuthz**, a public sample policy set for agentic AI authorization ([AWS Samples, 2026](#)). It is close to the AgentCore setting: Amazon Bedrock AgentCore Policy uses CEDAR to secure agent workflows and tool calls ([Hadarean and Tristan, 2026](#)). The sample models a tool call through three CEDAR policy layers:

Layer	Question it answers	Examples of facts it reads
L1	May this agent call this tool?	agent trust level, lifecycle, namespace, tool capability
L2	Was the delegation to this agent allowed?	delegating agent, delegated agent, requested capabilities, delegation depth
L3	May the originating user authorize this action?	user role, MFA status, requested action, delegation depth

A CEDAR request is allowed if a permit policy matches and no forbid policy matches (implicit deny and forbid overrides permit). In the example above, a deployed tool call is allowed only if all three policy layers allow it. (We note that this is not the same as concatenating all policies into one CEDAR policy set: a merged set would ask whether any layer contains a surviving permit.)

The implementation mirrors the deployment: it solves each layer as a separate CEDAR region question and intersects the layer results. For an `invoke_tool` request, the L2 check is not literally present in the policy text. The deployment reconstructs a `delegate_task` request from the tool-call context, carrying the same delegation depth and using the previous-hop agent as the delegating principal. Our model supplies that rewrite rule explicitly. SymCC proves the per-layer CEDAR claims; the cross-layer conjunction and the reconstructed L2 request are deployment logic.

The smallest example is the L2 sub-question: *when may the orchestrator delegate to the finance agent?* This isolates one free axis, `context.delegation_depth`. The finance-agent permit L2-001 applies when `delegation_depth <= 3` and the requested capabilities are available. The depth-limit policy L2-004 forbids any delegation request when `delegation_depth > 5`. The same policy layer also contains permits for other resources, such as data and support agents, but those policies are outside this question once the resource is fixed to `finance-agent`.

The desired answer is a partition over all depths, not a sample of a few depths:

Sub-region for <code>delegation_depth</code>	Verdict	Determining policy
<code>[0, 3]</code>	Allow	finance-agent permit L2-001
<code>[4, 5]</code>	Deny	implicit default deny: no permit applies
<code>[6, ∞)</code>	Deny	depth-limit forbid L2-004

The middle row is the useful bit of the example. Nothing special happens at depths 4 and 5 except that no permit applies and the depth-limit forbid has not yet fired. The system still reports that gap, because CEDAR’s default is deny. If the operator includes an unconditional catch-all forbid from the sample, the whole region becomes uniformly **Deny**. That is not a solver approximation; it is a different policy set, and the implementation reports the choice.

A.2. Using SymCC to partition a region

SymCC is a symbolic compiler that lets you check properties. It compares policy sets and can return counterexample requests when a property fails. We turn that into a region partitioner by encoding a candidate region R as a generated CEDAR guard policy G_R : a single **permit** whose **when** clause is exactly the conjunction of the region constraints. A request is allowed by G_R exactly when it lies in R .

Let P be the target policy set’s net allow decision under CEDAR’s usual semantics. The implementation asks SymCC two questions:

$$G_R \Rightarrow P \quad \text{and} \quad G_R \cap P = \emptyset.$$

If the implication holds, every request in R is allowed by P . If disjointness holds, no request in R is allowed by P . If both fail, the region contains both allowed and denied requests. SymCC returns witnesses for those cases, and the implementation *splits* the region into sub-regions and recurses on each. It repeats this process until each sub-region is proved uniform, marked as still dependent on an unspecified fact, or used to choose a clarifying question.

This is recursive case-splitting, not All-SAT enumeration. We never exclude models with blocking clauses or generate a **forbid**: the only synthesized policy is the single **permit** G_R . On a mixed region we pick one free axis and replace G_R with the guards of its sub-regions, each being G_R with one more conjunct in its **when** clause. These guards tile the parent region exactly (an integer axis into adjacent intervals, an enum into one sub-region per literal plus a none-of-these complement, a boolean into true/false, an opaque predicate into the clause and its negation), so the children partition R rather than sample it. Recursion stops at a depth cap or leaf budget, beyond which a region is reported as *it depends* rather than guessed.

The split axes come from CEDAR’s parsed policy representation. Integer comparisons produce cut-points; equality against strings or enums produces finite alternatives; booleans produce true/false cases; and datetime literals produce time bands. Richer expressions, such as entity membership, attribute presence, set predicates, or comparisons between attributes, are rendered back to valid CEDAR and treated as opaque predicates. The heuristic chooses which axis to try first; CEDAR still parses, type-checks, symbolically compiles, and authorizes every generated guard.

A.3. Contract and limits

The contract is the same as in the main paper. The symbolic layer chooses regions and proves uniformity; the unmodified engine supplies reported verdicts and citations. In the CEDAR implementation, a returned uniform branch is a theorem about every request accepted by its guard policy under CEDAR’s symbolic semantics. An interval such as $[6, \infty)$ denotes every relevant CEDAR integer in that range, not a representative grid point. The in-process CEDAR authorizer then runs on concrete witnesses or SymCC-produced counterexample environments to read off the branch’s *determining policies*, the policies CEDAR reports as the reason for its decision: the permit(s) that allowed the request, or the forbid(s) that denied it, one or more when several match. These are the citations in the rendered answer; a branch with no determining policy is an implicit default-deny.

The LLM is outside the trusted core. It helps lift a user utterance to a candidate region and verbalizes the result, but it does not decide any branch. If SymCC returns unknown, hits a resource limit, encounters a policy feature whose symbolic treatment is too coarse, or cannot refine a mixed region within the cost budget, the implementation marks the branch **It depends** or asks a clarifying question. It does not render that branch as uniformly allow or deny.

Using CEDAR’s symbolic compiler improves the policy proof, but it does not remove modeling choices. The answer is only about the policies included in the query, the CEDAR features SymCC can prove, and the deployment-composition rule supplied by the application.

A.4. Results

We evaluate three things: the motivating **AgentAuthz** regions, generality on unrelated CEDAR corpora, and a policy-visible LLM baseline that tries to produce the same conditional answer directly.

AgentAuthz. On twelve **AgentAuthz** question families, the core matches an independent CEDAR **Authorizer** gold on all twelve. The dataset covers originating-user policies, agent-to-tool policies, direct delegation policies, and two end-to-end tool-call questions that compose all three layers. In total, the gold contains 16,656 concrete request cells and 92 allow cells. Every symbolic answer is strict-correct against that gold, with mean per-cell accuracy 1.00 and determining-policy match 1.00 on allow cells.

Question group	Questions	Gold cells	Allow cells	Strict-correct
Originating-user layer	4	336	41	4/4
Agent-to-tool layer	3	168	10	3/3
Delegation layer	3	24	11	3/3
End-to-end layered	2	16,128	30	2/2
Total	12	16,656	92	12/12

Agent and policy-visible LLM comparison. We next evaluate the user-facing agent, not just the symbolic core. For OURS-AGENT, the model receives the natural-language question and must drive the same tools used in the main system: discover askable targets, inspect the free axes, lift the question to a region, call the symbolic region tool, and emit a JSON rule list. A separate trace check verifies that the tool calls select the intended action, concrete principal/resource ids, and pins. The emitted JSON rules are then expanded over the independent CEDAR **Authorizer** gold and scored deterministically. To isolate lift and answer quality from dialogue policy, the branch budget is raised high enough that no clarification question is needed on these questions.

We compare against two Claude Sonnet 4.5 policy-visible baselines, both at temperature zero, the Cedar analogues of B1 and B2 in the main paper. Both receive only the relevant policy text and the natural language question, and both must emit the same JSON rule schema as OURS-AGENT. LLM-LITERAL is told that every applicable layer must permit. LLM-TOLD additionally receives the out-of-policy-text L2 reconstruction rule and pinned

Table 2: Comparison on twelve **AgentAuthz** questions. OURS-AGENT is the actual LLM-lifted agent path; trace checks its tool route and lift. All systems emit structured JSON rules that are expanded against the independent CEDAR **Authorizer** gold; the determining-policy score is weighted by **Allow** cells.

System	Lift/tool trace	Strict-correct	Verdict acc.	Det. policy
OURS-AGENT	10/12	10/12	0.999	1.00
LLM-TOLD	–	7/12	0.888	1.00
LLM-LITERAL	–	6/12	0.888	0.67

capability sets, the same composition specification the symbolic core receives. We use one trial per question for each LLM arm.

Table 2 shows that the symbolic core’s accuracy does not, by itself, imply end-to-end agent accuracy. Expanded against the same gold, the core is strict-correct on 12/12 questions with determining-policy match 1.00; the user-facing agent is strict-correct on 10/12 after its LLM lift and final JSON generation are included.

The two OURS-AGENT failures are informative. Both are end-to-end tool-call questions. For *when is the finance agent authorized to process a payment?*, the lift pins **namespace=payments**. For *when is the data bot authorized to delete records?*, it pins **namespace=data**. Those namespace values are plausible from the entity and tool names, but they are not stated by the user. The region tool then answers the narrower pinned region correctly, and the final JSON rules cite the correct determining policies, but they omit namespace as an explicit condition. Scored against the intended open region, those rules over-allow other namespaces. This is the remaining source of potential unsoundness: the symbolic core is sound for the region it is given, but an LLM lift can silently make the region too small by pinning an unstated free attribute. The SMT encoding already supports detection: if freeing a pinned variable changes the partition, the system could surface the pin as a clarification question rather than accepting it silently. We note this as the natural next step for the lift guardrail described in the evaluation.

For auditability, every score is computed from a stored JSON rule list rather than from unstructured model prose.